DOCCHAT PROJECT

The RAG Stack â€” A Concept Guide

Every library, model, and pattern used in this project â€” explained clearly, from the ground up. Read it to understand what each piece does, why it exists, and how they connect into the finished app.

RAG | LangChain | ChromaDB | HuggingFace Embeddings | Ollama Â· Mistral | Streamlit | Python 3.12

CONTENTS

FOUNDATION

What is RAG?

01

RAG stands for **Retrieval-Augmented Generation**. It is a technique that makes a language model answer questions about documents it was never trained on â€” by first *retrieving* the relevant text, then *generating* an answer from it.

> ANALOGY
>
> Imagine giving an open-book exam. Instead of memorising the entire textbook (which is what a plain LLM does â€” it memorises its training data), you open the book, find the relevant paragraph, and write your answer. RAG is that open-book strategy for AI.

WHY NOT JUST USE AN LLM DIRECTLY?

A plain LLM like Mistral was trained on data up to a certain date. It knows nothing about *your* PDF. You could paste the whole PDF into the prompt, but:

- LLMs have a **context window limit** â€” most models can only read a few thousand words at once.
- Longer contexts slow the model down and cost more.
- The model can lose track of information buried deep in a long context ("lost in the middle" problem).

HOW RAG SOLVES THIS

RAG breaks the problem into two steps:

STEP 1 Â€" RETRIEVE

Convert the user's question into a vector. Search the document's pre-embedded chunks for the most similar ones. Pull only those (typically 3â€“5 chunks) into the context.

STEP 2 Â€" GENERATE

Feed those chunks + the original question to the LLM. It now has the exact relevant paragraphs right in front of it and can answer accurately.

The result: the LLM only ever sees a small, highly relevant slice of the document â€” exactly what it needs, nothing more.

ARCHITECTURE

02

The Full Pipeline

Your project runs two distinct pipelines. The **ingestion pipeline** runs once when a PDF is uploaded. The **query pipeline** runs every time the user sends a message.

INGESTION Â€" WHEN A PDF IS UPLOADED

PDF File uploaded → **PyPDFLoader** extract text → **Text Splitter** chunk it → **MiniLM** embed chunks → **ChromaDB** store vectors

QUERY Â€" WHEN THE USER ASKS A QUESTION

User Question → **MiniLM** embed question → **ChromaDB** similarity search → **Top-k Chunks** retrieved context → **Mis** generat

KEY INSIGHT

The same embedding model (`all-MiniLM-L6-v2`) is used in both pipelines. Chunks and questions are embedded in the same vector space, which is why similarity search works â€" the question vector lands near the chunk vectors that contain the answer.

DOCUMENT PROCESSING

03

PDF Loading â€” PyPDFLoader

`PyPDFLoader` is a LangChain wrapper around the `pypdf` library. It opens a PDF file, reads each page, and returns a list of `Document` objects â€” one per page.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("temp.pdf")
docs = loader.load()
# docs is a list of Document objects
# Each Document has:
#   .page_content  â€” the raw text of that page
#   .metadata      â€” {"source": "temp.pdf", "page": 0}
```

WHAT A DOCUMENT OBJECT CONTAINS

PAGE_CONTENT

The raw extracted text of the page. Quality depends on the PDF â€” text-based PDFs extract cleanly; scanned PDFs are images and need OCR (not handled here).

METADATA

A dictionary: `{"source": "temp.pdf", "page": 3}`. The page number is what your app shows in the "Sources" expander after each answer.

WHY SAVE TO TEMP.PDF FIRST?

Streamlit's `st.file_uploader` gives you an in-memory file object, not a path on disk. `PyPDFLoader` needs a file path, so you write the bytes to `temp.pdf` first, then pass that path to the loader.

DOCUMENT PROCESSING

04

Text Chunking â€” RecursiveCharacterTextSplitter

A 44-page PDF might contain 50,000 words. You can't embed the whole thing as one unit â€” you need to break it into small, searchable chunks. `RecursiveCharacterTextSplitter` does this intelligently.

> **ANALOGY**
>
> Think of it like cutting a long newspaper article into index cards. Each card has enough context to be useful on its own, and cards from the same paragraph share some overlap so meaning isn't lost at the seam.

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,     # max characters per chunk
    chunk_overlap=50    # characters shared between adjacent chunks
)
docs = splitter.split_documents(docs)
```

PARAMETERS EXPLAINED

PARAMETER	VALUE USED	WHAT IT MEANS
`chunk_size`	500	Each chunk will be at most 500 characters (~80â€“100 words). Small enough for the embedding model to encode accurately; large enough to hold a complete thought.
`chunk_overlap`	50	The last 50 characters of chunk N are repeated at the start of chunk N+1. This prevents answers from being cut off when the relevant information straddles a boundary.

WHY "RECURSIVE"?

The splitter tries to split on natural boundaries in order: `"\n\n"` (paragraphs) â†’ `"\n"` (lines) â†’ `" "` (words) â†’ characters. It only uses the next separator if the chunk is still too big. This preserves sentence structure wherever possible.

MACHINE LEARNING

05

Text Embeddings â€” all-MiniLM-L6-v2

An **embedding** is a list of numbers (a vector) that represents the *meaning* of a piece of text. Two pieces of text with similar meanings will have vectors that are close together in space; unrelated texts will have vectors far apart.

> ANALOGY
>
> Imagine placing every sentence on a giant map where meaning determines location. "The dog chased the cat" and "A puppy ran after the kitten" land near each other on this map. "The stock market fell" lands far away. Embeddings are the GPS coordinates on that map.

THE MODEL: ALL-MINILM-L6-V2

This is a lightweight sentence-transformer model from HuggingFace. It converts any text into a **384-dimensional vector** (a list of 384 decimal numbers). It was trained specifically for *semantic similarity* tasks â€” meaning it's optimised to group similar-meaning text close together.

PROPERTY	VALUE	WHY IT MATTERS
`Output size`	384 dimensions	Small enough to be fast, large enough to be expressive
`Model size`	~23 MB	Downloads and loads quickly; runs on CPU
`Max input`	256 tokens (~200 words)	Reason chunk_size=500 chars works: most English chunks stay under this
`Runs locally`	Yes	No API key, no internet required after first download

```
from langchain_community.embeddings import HuggingFaceEmbeddings

embedding = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

# Embed a single text (returns a list of 384 floats)
```

```
vector = embedding.embed_query("What is attention mechanism?")
print(len(vector))  # â†’ 384
```

SIMILARITY SEARCH

When a user asks a question, it gets embedded into the same 384-dimensional space. ChromaDB then finds the stored chunk vectors that are *closest* to the question vector using **cosine similarity** â€” measuring the angle between vectors rather than their distance, which is more robust for text.

STORAGE

Vector Database â€” ChromaDB

06

A **vector database** is a database built specifically to store and search vectors efficiently. Unlike a regular database that searches by exact match (SQL `WHERE name = 'Alice'`), a vector database searches by *similarity* â€” "find me the 4 vectors closest to this query vector."

> ANALOGY
>
> A regular database is like looking up a word in a dictionary â€” you need the exact spelling. A vector database is like a "sounds like" search â€” it finds things that are conceptually close even if the words are different.

CHROMADB IN YOUR PROJECT

```
# Ingestion: create collection from documents
vectordb = Chroma.from_documents(
    documents=docs,
    embedding=embedding,
    persist_directory="db"   # saved to disk in ./db/
)

# Query: load the saved collection
vectordb = Chroma(
    persist_directory="db",
    embedding_function=embedding
)
retriever = vectordb.as_retriever()  # retrieves top-4 chunks by
default
```

WHAT'S STORED IN THE `./DB/` FOLDER?

CHROMA.SQLITE3

A SQLite database file that stores metadata, collection configurations, and the text content of each chunk alongside its vector.

HNSW INDEX

The actual vector index â€” a data structure (Hierarchical Navigable Small World graph) that enables fast approximate nearest-neighbour search even over millions of vectors.

WHY NOT JUST USE NUMPY?

You could compute cosine similarity over all vectors with NumPy. For 100 chunks it's fine. But for a 500-page document with 2,000 chunks, or across multiple documents, a proper index is orders of magnitude faster. ChromaDB also persists data to disk so you don't re-embed on every restart.

LANGUAGE MODEL

07

Ollama & Mistral

Ollama is a tool that lets you run large language models locally on your own machine. It handles downloading models, managing their memory, and serving them through a local HTTP API at `http://localhost:11434`. Your Python code never needs an API key or internet connection when talking to Ollama.

WHY RUN LOCALLY?

PRIVACY

Your PDF content never leaves your machine. Crucial for confidential documents â€” financial reports, legal contracts, medical records.

COST

No per-token charges. You can run millions of tokens for free (just electricity and hardware). Cloud APIs like OpenAI charge per use.

CONTROL

The model never changes under you. No API deprecations, no model updates that alter behaviour, no rate limits.

OFFLINE

Works with no internet after first download. Useful for air-gapped environments or travel.

MISTRAL 7B

Mistral 7B is an open-weight LLM developed by Mistral AI (France). "7B" means it has 7 billion parameters. Key properties:

- **Instruction-following:** it understands prompts like "Answer the question based only on the context below."
- **Context window:** 32,768 tokens â€” more than enough for the 3â€“5 retrieved chunks plus the question.
- **Speed:** On a modern CPU, a typical answer takes 5â€–30 seconds. With a GPU, under 5 seconds.
- **Quality:** Competitive with GPT-3.5 on many benchmarks, especially instruction-following and reasoning tasks.

```python
from langchain_community.llms import Ollama

llm = Ollama(model="mistral")
# Ollama must be running: `ollama serve`
# Model must be downloaded: `ollama pull mistral`
# LangChain sends the prompt to http://localhost:11434
```

FRAMEWORK

08

LangChain

LangChain is a Python framework for building applications powered by language models. It provides standard interfaces and pre-built "chains" so you don't have to write the glue code yourself. Your project uses three LangChain packages:

PACKAGE	WHAT IT PROVIDES IN YOUR PROJECT
`langchain`	Core abstractions: `Document`, `Chain`, `RetrievalQA`
`langchain-community`	Integrations: `PyPDFLoader`, `HuggingFaceEmbeddings`, `Ollama`, `Chroma`
`langchain-text-splitters`	`RecursiveCharacterTextSplitter`

CORE CONCEPT: THE CHAIN

A LangChain "chain" is a reusable pipeline of steps. You define each step once, then call the chain with an input and it runs all steps in order. Chains are composable â€” you can plug them together to build complex workflows.

> **ANALOGY**
>
> Think of a chain like a Unix pipe: `cat file.txt | grep keyword | sort | uniq`. Each command does one thing; they compose into a larger operation. LangChain chains work the same way â€” each link transforms the data and passes it to the next.

WHY USE LANGCHAIN AT ALL?

Without LangChain, you'd write code to: load the PDF, split text, call the embedding model, talk to the vector DB, format a prompt with context, call the LLM, and parse the output. LangChain packages all of this behind standard interfaces, making it trivial to swap components â€” change `Ollama` to `OpenAI`, or `Chroma` to `Pinecone`, with a one-line change.

FRAMEWORK

RetrievalQA Chain

09

`RetrievalQA` is a LangChain chain that combines a *retriever* (finds relevant chunks) with a *language model* (generates an answer). It's the central piece that makes RAG work.

```
qa = RetrievalQA.from_chain_type(
    llm=llm,              # the Mistral model
    retriever=retriever,  # ChromaDB similarity search
    return_source_documents=True  # also return which chunks were used
)

response = qa.invoke({"query": "What is the attention mechanism?"})
answer  = response["result"]           # the generated text
sources = response["source_documents"] # list of retrieved Document
objects
```

WHAT HAPPENS INSIDE `QA.INVOKE()`

1. Your question is passed to the **retriever**, which embeds it and searches ChromaDB for the top-4 most similar chunks.
2. The chain formats a **prompt**: *"Use the following context to answer the question. Context: [chunk1] [chunk2] [chunk3] [chunk4]. Question: [your question]. Answer:"*
3. This prompt is sent to **Mistral** via Ollama.
4. Mistral's generated response is returned as `result`. The retrieved chunks are returned as `source_documents`.

CHAIN_TYPE

`from_chain_type` defaults to `chain_type="stuff"` â€” all retrieved chunks are "stuffed" into a single prompt. Alternative types like `map_reduce` (process each chunk separately, then combine) exist for longer documents that exceed the model's context window.

SOURCE DOCUMENTS IN YOUR APP

The `source_documents` list is what your app uses to show the "Sources â€” N page(s)" expander. Each element is a LangChain `Document` with `.metadata["page"]` giving the original PDF page number.

USER INTERFACE

10

Streamlit

Streamlit is a Python library that turns a Python script into an interactive web app. Every time a user interacts (clicks a button, types in the chat, uploads a file), Streamlit **reruns the entire script from top to bottom**. This is the most important thing to understand about Streamlit.

> ANALOGY
>
> A Streamlit app is like a spreadsheet: when you change one cell, everything recalculates. When you interact with any widget, Streamlit recalculates â€” re-executes â€” the entire app. State is managed explicitly, not implicitly.

KEY STREAMLIT CONCEPTS USED IN YOUR APP

API	WHAT IT DOES
`st.chat_input()`	A fixed-at-bottom text input that triggers a rerun when submitted. Returns the typed text or `None`.
`st.chat_message(role)`	A styled container for one message. `role="user"` or `role="assistant"` controls the avatar and layout.
`st.sidebar`	A context manager that routes any widget inside it to the left sidebar panel.
`st.file_uploader()`	A drag-and-drop file widget. Returns an `UploadedFile` object (bytes + filename) or `None`.
`st.spinner()`	A "loading" context manager â€” shows a spinner while the code block inside runs.
`st.expander()`	A collapsible section â€” used for the Sources panel.
`st.rerun()`	Force an immediate full re-run of the script. Used after clearing chat history.
`st.session_state`	A dictionary that persists between reruns. The only way to store state across interactions.

WHY SESSION_STATE MATTERS

Without `st.session_state`, every rerun would forget everything â€” chat history, the QA chain, which PDF was loaded. Your app uses three session state keys:

```
st.session_state.messages  # list of {role, content, sources} dicts
st.session_state.qa        # the built RetrievalQA chain (expensive to
rebuild)
st.session_state.pdf_name  # filename of the last uploaded PDF
```

ENGINEERING

Performance Patterns

11

The app uses three specific Python/Streamlit patterns to stay fast. Understanding them helps you extend the project without accidentally re-introducing slowness.

PATTERN 1 Â€" LAZY IMPORTS

Heavy packages (`torch`, `langchain_community`, `chromadb`) take 10â€"40 seconds to import on first load. By moving these imports *inside* the functions that need them, the page loads instantly. Python's module cache (`sys.modules`) ensures each package is only imported once, regardless of how many times the function is called.

```
# Bad â€" runs on every page load (20+ seconds)
from langchain_community.vectorstores import Chroma  # at top of file

# Good â€" runs only when process_pdf() is first called
def process_pdf(path):
    from langchain_community.vectorstores import Chroma  # inside
function
    ...
```

PATTERN 2 Â€" @ST.CACHE_RESOURCE

This decorator tells Streamlit to run a function *once* and cache the result in memory. Subsequent calls return the cached result instantly. It is specifically designed for expensive-to-build, stateful objects like ML models and database connections.

```
@st.cache_resource(show_spinner=False)
def _load_embedding_model():
    from langchain_community.embeddings import HuggingFaceEmbeddings
    return HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")

# First call: loads model from disk (~5s)
# Every subsequent call: returns immediately from cache
```

CACHE_RESOURCE VS CACHE_DATA

`@st.cache_resource` caches stateful objects (models, DB connections) â€" shared across all users and sessions. `@st.cache_data` caches pure functions (serialisable

outputs like dataframes, JSON) â€” isolated per user. Use `cache_resource` for anything that holds a connection or loaded weights.

PATTERN 3 Â€” SESSION STATE FOR THE QA CHAIN

The QA chain (`st.session_state.qa`) is rebuilt only when a *new* PDF is uploaded, detected by comparing filenames:

```
if st.session_state.pdf_name != uploaded_file.name:
    # New PDF â€” rebuild the chain
    st.session_state.qa = process_pdf("temp.pdf")
    st.session_state.pdf_name = uploaded_file.name
# Same PDF â€” chain is already in session_state, skip processing
```

Without this guard, every chat message would trigger PDF reprocessing (30+ seconds per question).

REFERENCE

12

Glossary

RAG

Retrieval-Augmented Generation. A technique where relevant documents are fetched first, then fed to an LLM as context for generating an answer.

Vector Database

A database optimised for storing and similarity-searching vectors. ChromaDB is the one used here, persisted locally in the `./db` folder.

Chunk

A small piece of text, typically 300â€“600 characters, produced by splitting a larger document. Each chunk is embedded and stored as one entry in ChromaDB.

Ollama

A tool for running LLMs locally. Serves models via an HTTP API at `localhost:11434`. Handles GPU/CPU memory management.

Token

The unit LLMs process text in â€” roughly Â¾ of a word. "hamburger" is 3 tokens. Pricing and context limits are measured in tokens, not words.

chain_type="stuff"

All retrieved documents are "stuffed" into a single prompt. Simple and fast. Alternatives: `map_reduce`, `refine`, `map_rerank`.

HNSW

Hierarchical Navigable Small World â€” the graph algorithm ChromaDB uses to find nearest-neighbour vectors in sub-linear time.

Persistent Client

ChromaDB's mode where data is saved to disk (the `./db` folder). Auto-persists on every

Embedding

A fixed-length list of numbers (a vector) that represents text meaning. Produced by a neural network trained on semantic similarity.

Cosine Similarity

A measure of similarity between two vectors based on the angle between them. A score of 1.0 = identical direction; 0.0 = orthogonal (unrelated).

LLM

Large Language Model. A neural network trained on vast text data that can generate human-like text. Mistral 7B is the LLM in this project.

Context Window

The maximum number of tokens an LLM can process at once. Mistral's is 32,768 tokens (~24,000 words). RAG ensures we stay well within this.

Retriever

A LangChain interface that wraps a vector store and implements `get_relevant_documents(query)`. By default returns the top-4 most similar chunks.

Session State

Streamlit's mechanism for persisting data across reruns. Stored in `st.session_state` as a dictionary. Lost when the browser tab closes.

all-MiniLM-L6-v2

A 23 MB sentence-transformer model from HuggingFace. Outputs 384-dimensional vectors. "L6" = 6 transformer layers. Fast, accurate, CPU-friendly.

Lazy Import

Moving a heavy `import` statement inside a function rather than at the top of the file, so

write â€” no need to call `.persist()` manually.

the module only loads when that function is first called.
